

Catálogo

Miguel Rodríguez Penabad

Laboratorio de Bases de Datos

Universidade da Coruña



UNIVERSIDADE DA CORUÑA

2021



- 1 Introducción
- 2 O catálogo do sistema
- 3 O catálogo SQL estándar
- 4 Casos de uso

Introdución

A necesidade dos metadatos

Un sistema de información plasma habitualmente coñecemento sobre un dominio. Por exemplo, unha organización e os seus procesos.

Inicialmente, estes sistemas estaban formados por ficheiros e programas que os manexaban. A estrutura e descrición dos datos do dominio estaba no código dos programas (ex: rexistros).

Se usásemos esta metodoloxía:

- Varios programas poden conter información sobre os datos incompatible.
- Non existe documentación do significado dos datos, de permisos (quen pode facer que sobre cada dato), procesos que os manexan ...

É evidente que esta metodoloxía é pouco útil e pouco mantible.

Nexte contexto aparecen o concepto de **dicionario de datos**: un repositorio centralizado que contén información sobre os datos (habitualmente agora almacenados nunha base de datos).

Non debe confundirse o dicionario de datos co **catálogo** dunha base de datos relacional. Aínda que este tamén contén metadatos, será usado tanto polo SXBD como polos usuarios, pero normalmente con outros obxectivos, e *non* pode ser modificado directamente polos usuarios.

- Un **Diccionario de Datos (DD)** é un repositorio de información que almacena metadatos (datos sobre os propios datos).
- Inclúe información sobre os propios datos (significado, tipos de datos, formatos, tamaños, restricións, ...), pero pode tamén incorporar outros elementos, como os programas que os utilizan ou usuarios/persoas que poden acceder a eles.
- O DD xestiona a información de forma centralizada (será consultado e/ou modificado por usuarios cos permisos necesarios), o que pode mellorar a autenticidade da información e a comunicación entre os usuarios.

Un DD permite:

- Documentar o modelo: requisitos, especificacións, deseño, ...
- Xenerar informes sobre calquera elemento do modelo (incluíndo elementos relacionados).
- Analizar o impacto dun cambio no modelo de datos (cambios de tipos de datos, inclusión de novos elementos, ...).
- Xenerar automaticamente a estrutura da base de datos (ex: sentencias SQL `create table`).
- Xenerar automaticamente a definición de datos en linguaxes de programación que accedan a eses datos (ex: clases Java).

Introdución

O catálogo do sistema

O **catálogo do sistema** dun SXBD relacional é un conxunto de táboas que almacenan metadatos sobre a base de datos, e vistas que expoñen estes datos de forma máis amigable.

É frecuente denominalas (de forma informal) como táboas e vistas do sistema, para diferenciarlas das táboas e vistas “dos usuarios”.

O catálogo é creado polo SXBD e mantido por el, pero é de só lectura para os usuarios.

Os usuarios poden “provocar” modificacións no catálogo, por exemplo mediante operacións DDL (ex: creando unha táboa).

O propio SXBD utiliza continuamente o catálogo para realizar as súas funcións.

O catálogo é crítico para o funcionamento do sistema.

1 Introducción

2 O catálogo do sistema

3 O catálogo SQL estándar

4 Casos de uso

O catálogo do sistema

Exemplos iniciais

O **catálogo** dun SXBD relacional pode ser utilizado tanto por un usuario como polo propio SXBD.

Consultas de usuario

Como podo saber as columnas da táboa EMP? (Consulta aproximada usando o catálogo estándar de SQL)

```
select column_name, data_type
  from information_schema.columns
 where table_name='EMP';
```

Uso por parte do sistema

Que pasa cando executo a seguinte consulta?

```
select *
  from emp
 where sal + comm > 2000
    and deptno=10;
```

- O SXBD comproba que emp exista e sexa unha táboa ou vista.
- Comproba que o usuario ten permiso de select sobre emp.
- Expande o * á lista de columnas da táboa.
- Comproba que as columnas no where existen e son do tipo axeitado (ex: numéricos para sumar)
- No proceso de optimización: Comproba se existen índices, identifica os espazos físicos de almacenamento, ...
- Etc.

Para obter toda esta información, o SXBD consulta o catálogo do sistema.

O catálogo do sistema

Contido do catálogo

O catálogo dun SXBD relacional almacena datos dos esquemas conceptual, externo e físico (según define o estándar ANSI/X3/SPARC).

Entre outros elementos, contén:

- Definición das táboas e as súas columnas
- As restricións de integridade (claves primarias, foráneas, ...)
- Definición das vistas
- Descrición dos espazos de almacenamento (ficheiros, espazos para táboas, ...)
- Índices asociados ás táboas
- Información sobre usuarios, roles e privilexios
- Descrición das estruturas lóxicas e físicas da BD

Ademáis, o catálogo:

- Autocontén os metadatos sobre a propia estrutura do catálogo.
- *Non contén* información externa á base de datos, como a definición (significado) dos datos, para que se usen, ou os programas que os manipulan.

1 Introducción

2 O catálogo do sistema

3 O catálogo SQL estándar

4 Casos de uso

O catálogo SQL estándar

Estrutura

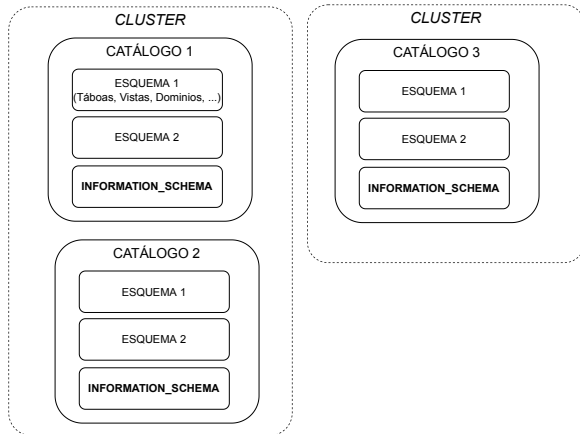
Según o estándar SQL, o catálogo sigue a seguinte estrutura:

1. **Esquema:** É unha colección de elementos individuais como táboas, vistas, triggers etc.
Un esquema pertence a un só usuario (en xeral, a un *ID de autorización*).
Un usuario ou ID de autorización pode ser o propietario de varios esquemas.
2. **Catálogo:** É un conxunto de esquemas (os nomes de esquema deben ser únicos nun catálogo).
Nomenclatura única dos elementos da BD: `nome_catalogo.nome_esquema.nome_elemento`.
Todo catálogo debe conter un esquema de nome `INFORMATION_SCHEMA`, que contén a información sobre todos os esquemas (e os elementos que inclúen) nese catálogo.
3. **Cluster:** É unha colección de catálogos.
Cada usuario ten asociado un cluster, que representa o conxunto de catálogos ós que ten acceso.
Un cluster é o máximo ámbito sobre o que se pode executar unha consulta SQL, polo que de algún xeito sería “a base de datos” tal como a ve o usuario.
4. **Entorno SQL:** (SQL Environment) é o contexto no que os datos poden existir e no que se poden realizar operacións sobre eles.
Na práctica podemos consideralo unha instancia do Sistema Xestor de Bases de Datos (SXBD) que se está executando nunha instalación concreta.

O catálogo SQL estándar

Visión global

ENTORNO (Instalación)



O catálogo SQL estándar

Almacenamento e exposición de metadatos

- O SXBD debe almacenar os metadatos nun conxunto de táboas base denominado `DEFINITION_SCHEMA`, que non serán accesibles polo SQL dos usuarios. Estas “táboas” son hipotéticas, xa que o estándar deixa esta parte aberta a diferentes implementacións.
- Os datos expóñense a través dunha colección de vistas no `INFORMATION_SCHEMA`.
 - `SCHEMATA`: Esquemas propiedade do usuario actual (o que executa a consulta sobre esta vista).
 - `TABLES`: Táboas persistentes accesibles para o usuario actual.
 - `COLUMNS`: Columnas das táboas persistentes accesibles para o usuario actual.
 - `VIEWS`: Vistas accesibles para o usuario actual.
 - `DOMAINS`: Dominios accesibles para o usuario actual.
 - ...

`INFORMATION_SCHEMA` é autodescriptivo: Contén información sobre as súas propias vistas en `INFORMATION_SCHEMA.VIEWS`.

- Os usuarios poden *consultar* o catálogo utilizando SQL, pero *non o poden modificar* directamente. As modificacións no catálogo son habitualmente consecuencia de operacións DDL.

- 1 Introducción
- 2 O catálogo do sistema
- 3 O catálogo SQL estándar
- 4 Casos de uso**

Oracle non sigue o estándar.

A estrutura dun entorno ou instalación de Oracle ten os seguintes elementos:

- Ata a versión 11, unha única base de datos. A partir da versión 12, instalando como *Container Database, CDB*), pode ter varias.
- Aínda que hai usuarios comúns, o habitual é que cada base de datos teña os seus usuarios propios.
- Cada usuario é propietario dun único esquema (que ten o mesmo nome).
- Dentro dun esquema hai elementos de distintos tipos. Oracle refírese a estes elementos de forma global como obxectos (*objects*)
 - Táboas, vistas
 - Secuencias, índices
 - Sinónimos
 - Paquetes, procedementos, funcións, disparadores
 - ...

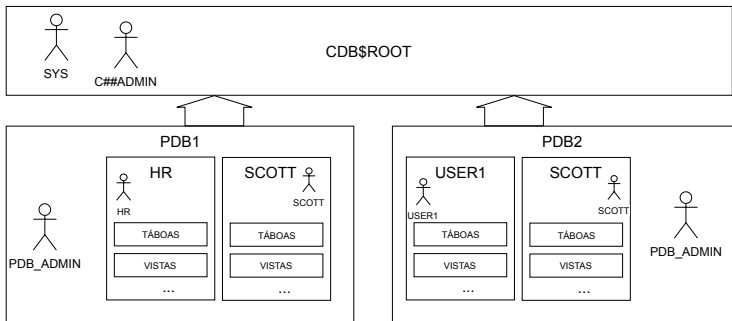
O nome único dun elemento (dentro da base de datos á que un usuario se conecta) será o nome do usuario (ou esquema) e o nome do obxecto unidos por un punto: `SCOTT.EMP`, por exemplo.

Casos de uso

Oracle

Oracle *Multitenant* (a partir da versión 12):

- Nunha mesma instalación podemos *enchufar* PDBs (Pluggable Databases).
- Desde o punto de vista do usuario/aplicación, unha PDB é moi similar a unha instalación dunha BD Oracle 11.



Para cada tipo de obxecto (en plural), en xeral, existen vistas cos seguintes prefixos:

- `USER_<tipo obxecto>`: Obxectos creados polo usuario.
- `ALL_<tipo obxecto>`: Obxectos ós que o usuario ten acceso (permisos).
- `DBA_<tipo obxecto>`: Todos os obxectos.
As vistas con este prefixo só poden ser consultados por usuarios co role DBA.
- `CDB_`: Todos os obxectos (de todas as PDBs se se consulta desde o container `CDB$ROOT`).
Só en Oracle 12 ou superior, con instalación como CDB (Container Database).
Tamén require privilexios de DBA.

Os tipos de obxecto que Oracle manexa son, entre outros:

- `OBJECTS` (Todos os tipos de obxecto)
- `TABLES`
- `VIEWS`
- `INDEXES`
- `CONSTRAINTS`
- `SYNONYMS`

Exemplos de vistas do catálogo:

- **DICTIONARY**: Describe brevemente todas as vistas do catálogo.
- **DICT**: Sinónimo de **DICTIONARY**.
- **USER_OBJECTS**: Todos os obxectos creados polo usuario.
- **ALL_VIEWS**: Vistas accesibles ó usuario.
- **DBA_INDEXES**: Todos os índices (só para DBAs).

Para a consulta dos elementos creados polo propio usuario son interesantes:

- **OBJ** (sinónimo de **USER_OBJECTS**).
- **CAT** (**USER_CATALOG**): Táboas, vistas e secuencias do usuario.
- **TABS** (**USER_TABLES**): Táboas do usuario.
- **COLS** (**USER_TAB_COLUMNS**): Columnas das táboas do usuario.
- **TAB**: Non confundir con **TABS**. Táboas e vistas do usuario.
- **USER_CONSTRAINTS**: Restricións (das táboas) do usuario.
- **USER_CONS_COLUMNS**: Columnas implicadas nas restricións (das táboas) do usuario.

Casos de uso

PostgreSQL

PostgreSQL sigue o estándar en certa forma, aínda que a nomenclatura varía.

<i>SQL Estándar</i>	<i>PostgreSQL</i>
Esquema	Schema (Esquema)
Catálogo	Databse (Base de datos)
Cluster	–
Entorno SQL	Cluster (Instalación)

- Un usuario pode crear múltiples bases de datos, e unha base de datos é propiedade dun só usuario.
- Un usuario pode acceder a bases de datos creadas por outros usuarios, pero non existe un nome para ese conxunto de bases de datos ás que pode acceder (o *cluster* do SQL estándar).
- Dentro dunha base de datos pode haber varios *Esquemas*. Un usuario pode crear varios esquemas nunha base de datos, e un esquema é dun só usuario.
- Cada base de datos contén un esquema de nome `INFORMATION_SCHEMA` (ademáis do propio `pg_catalog`).

Casos de uso

PostgreSQL

- ▼ penabad@localhost
 - ▼ Databases (5)
 - ▼ docencia
 - > Casts
 - ▼ Catalogs (2)
 - > ANSI (information_schema)
 - > PostgreSQL Catalog (pg_catalog)
 - > Event Triggers
 - > Extensions
 - > Foreign Data Wrappers
 - > Languages
 - > Publications
 - ▼ Schemas (4)
 - > abd
 - > bda
 - > mai
 - > public
 - > Subscriptions
 - > investigacion
 - > penabad
 - > postgres

Casos de uso

PostgreSQL

Exemplo de creación de esquemas e táboas:

```
create schema abd;
create table abd.estudiante(
    dni varchar(9),
    apellidos varchar(80) not null,
    nome varchar(40) not null,
    login varchar(50) not null,
    constraint pk_estudiante primary key(dni),
    constraint u_login_est unique(login)
);
```

```
create schema bda;
create table bda.estudiante(
    dni varchar(9),
    apellidos varchar(80) not null,
    nome varchar(40) not null,
    login varchar(50) not null,
    mencion varchar(3) not null,
    constraint pk_estudiante primary key(dni),
    constraint u_login_est unique(login),
    constraint ch_mencion check (mencion in ('ES','SI'))
);
```

```
create schema mai;
create table mai.estudiante(
    dni varchar(9),
    apellidos varchar(80) not null,
    nome varchar(40) not null,
    login varchar(50) not null,
    constraint pk_estudiante primary key(dni),
    constraint u_login_est unique(login)
);
```

Casos de uso

PostgreSQL

Consulta utilizando o estándar INFORMATION_SCHEMA e o propio de PostgreSQL pg_catalog.

```
docencia=> select table_schema, table_name, table_type
            from information_schema.tables
            where table_schema in ('abd','bda','mai');
```

table_schema	table_name	table_type
abd	estudiante	BASE TABLE
mai	estudiante	BASE TABLE
bda	estudiante	BASE TABLE

```
docencia=> select schemaname, tablename
            from pg_catalog.pg_tables
            where schemaname in ('abd','bda','mai');
```

schemaname	tablename
abd	estudiante
mai	estudiante
bda	estudiante

A primeira consulta funcionaría en calquera sistema que utilizase o estándar INFORMATION_SCHEMA (por exemplo, MySQL/MariaDB ou versións recentes de Microsoft SQL Server).

Bibliografía



Hector Garcia-Molina, Jeffrey D. Ullman, and Jennifer Widom.

Database Systems: The Complete Book.

Prentice Hall Press, Upper Saddle River, NJ, USA, 2 edition, 2008.



J. Melton and A. R. Simon.

SQL:1999 - Understanding relational language components.

Morgan Kaufmann, 2002.